

Week 8 Problem Set

Self-adjusting Search Trees

1. (Insertion at root)

a. Consider an initially empty BST and the sequence of values

1 2 3 4 5 6

- Show the tree resulting from inserting these values "at leaf". What is its height?
- Show the tree resulting from inserting these values "at root". What is its height?
- Show the tree resulting from alternating between at-leaf-insertion and at-root-insertion. What is its height?

b. Complete this week's Binary Search Tree ADT ([BST.h](#), [BST.c](#)) from the lecture by an implementation of the function:

```
Tree insertAtRoot(Tree t, Item it) { ... }
```

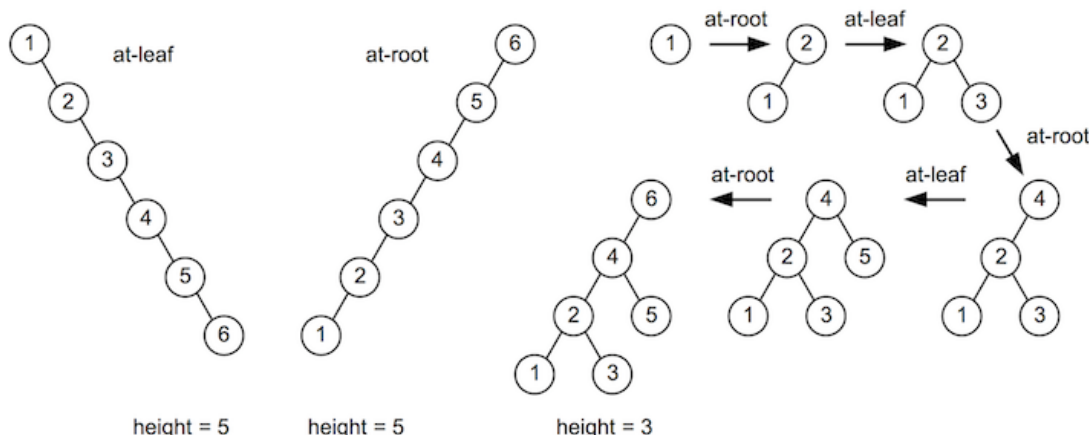
We have created a script that can automatically test your program. To run this test you can execute the dryrun program that corresponds to this exercise. It expects to find the file named `BST.c` in the current directory with your implementation of the function `insertAtRoot()`.

You can use dryrun as follows:

```
prompt$ 9024 dryrun BST
```

Answer:

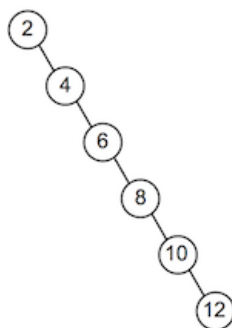
- a. At-leaf-insertion results in a "right-deep" tree while at-root insertion results in a "left-deep" tree. Both are fully degenerate trees of height 5. Alternating between the two styles of insertion results in a tree of height 3. Generally, if n ordered values are inserted into a BST in this way, then the resulting tree will be of height $\left\lfloor \frac{n}{2} \right\rfloor$.



- b.
- ```
Tree insertAtRoot(Tree t, Item it) {
 if (t == NULL) {
 t = newNode(it);
 } else if (it < data(t)) {
 left(t) = insertAtRoot(left(t), it);
 t = rotateRight(t);
 } else if (it > data(t)) {
 right(t) = insertAtRoot(right(t), it);
 t = rotateLeft(t);
 }
 return t;
}
```

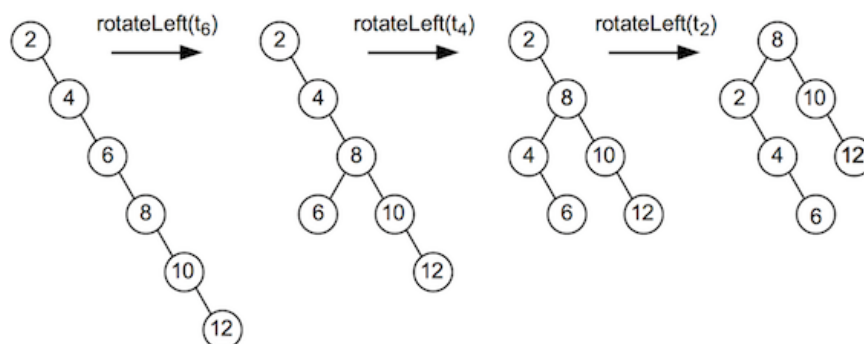
#### 2. (Rebalancing)

Trace the execution of `rebalance(t)` on the following tree. Show the tree after each rotate operation.

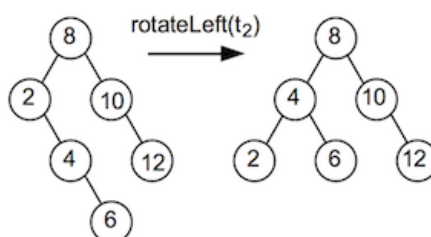
**Answer:**

In the answer below, any (sub-)tree  $t_n$  is identified by its root node  $n$ , e.g.  $t_2$  for the original tree.

Rebalancing begins by calling `partition( $t_2, 3$ )` since the original tree has 6 nodes. The call to `partition( $t_2, 3$ )` leads to a series of recursive calls: `partition( $t_4, 2$ )`, which calls `partition( $t_6, 1$ )`, which in turn calls `partition( $t_8, 0$ )`. The last call simply returns  $t_8$ , and then the following rotations are performed to complete each recursive call:



Next, the new left subtree  $t_2$  gets balanced via `partition( $t_2, 1$ )`, since this subtree has 3 nodes. Calling `partition( $t_2, 1$ )` leads to the recursive call `partition( $t_4, 0$ )`. The latter returns  $t_4$ , and then the following rotation is performed to complete the rebalancing of subtree  $t_2$ :



The left and right subtrees of  $t_4$  have fewer than 3 nodes, hence will not be rebalanced further. Rebalancing continues with the right subtree  $t_{10}$ . Since this tree also has fewer than 3 nodes, rebalancing is finished.

## 3. (Splay trees)

- a. Show how a Splay tree would be constructed if the following values were inserted into an initially empty tree in the order given:

3 8 5 7 2 4

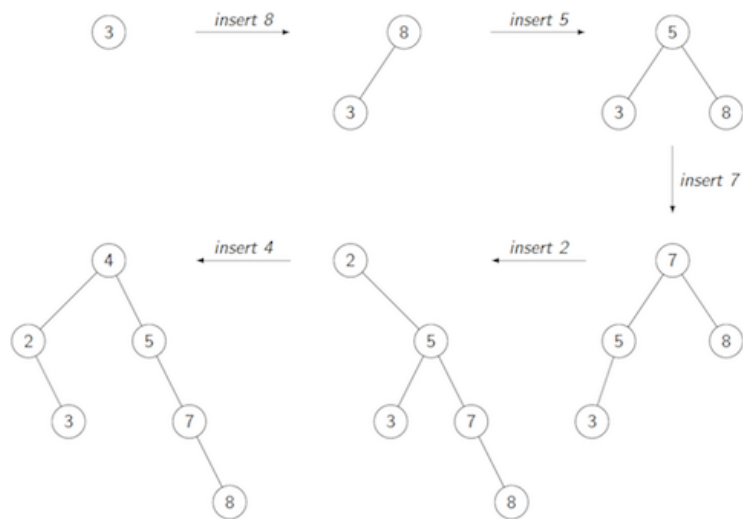
- b. Let  $t$  be your answer to question a., and consider the following sequence of operations:

```
SearchSplay($t, 3$)
SearchSplay($t, 5$)
SearchSplay($t, 6$)
```

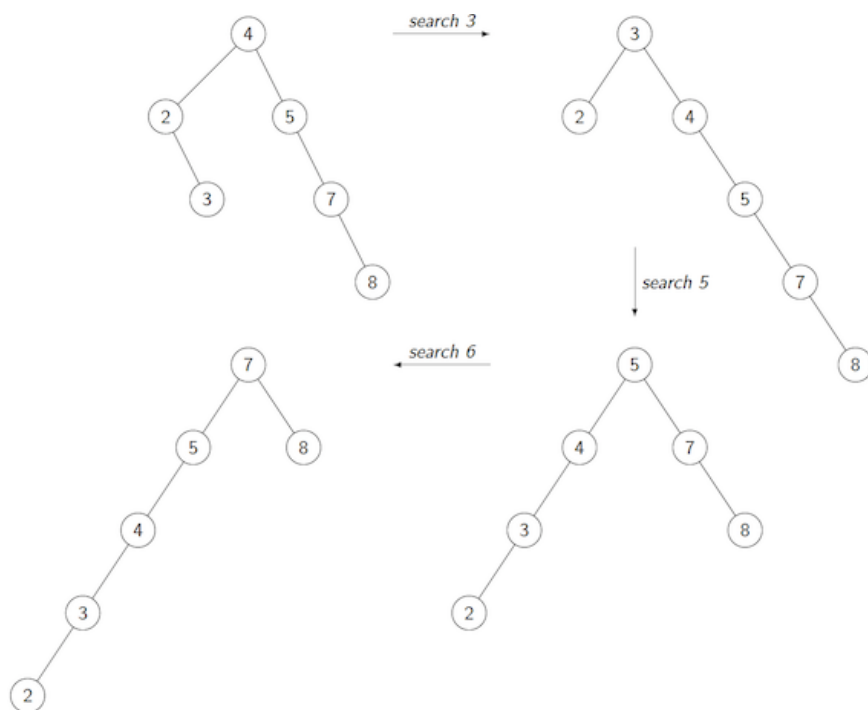
Show the tree after each operation.

**Answer:**

- a. The following diagram shows how the tree grows:



b. The following diagram shows how the tree changes with each search operation:



#### 4. (AVL trees)

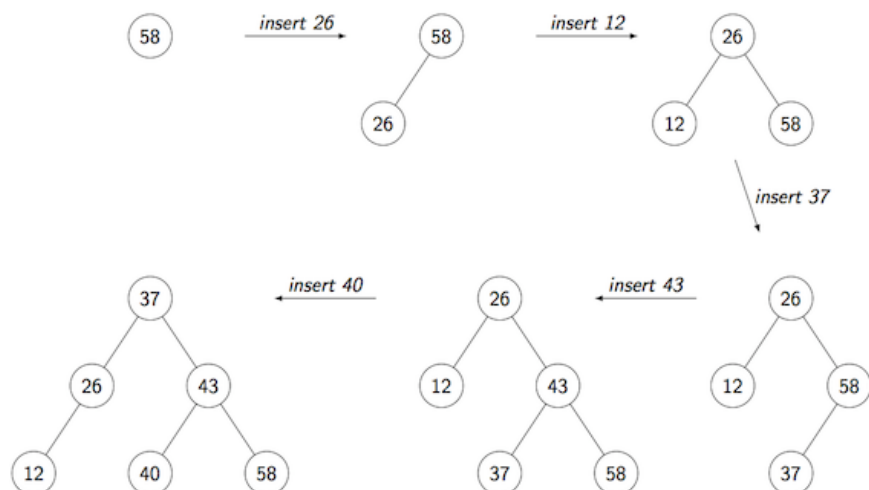
Note: You should answer the following question without the help of the treeLab program from the lecture.

Show how an AVL tree would be constructed if the following values were inserted into an initially empty tree in the order given:

58 26 12 37 43 40

**Answer:**

The following diagram shows how the tree grows:



Imbalances happen:

- when 12 is inserted, which triggers a right rotation at the root;
- when 43 is inserted, which triggers a left rotation at 37 followed by a right rotation at 58;
- when 40 is inserted, which triggers a right rotation at 43 followed by a left rotation at the root.

#### 5. (Lazy deletion)

There are (at least) two approaches to dealing with deletions from binary search trees. The first, as used in lectures, is to remove the tree node containing the deleted value and re-arrange the pointers within the tree. The second is to not remove nodes, but simply to mark them as being "deleted".

For this question, assume that we are going to re-implement Binary Search Trees so that they use mark-as-deleted rather than deleting any nodes. Under this scheme, no nodes are ever removed from the tree; instead, when a value is deleted, its node remains (and continues to retain the same value) but is marked so that it can be recognised as deleted.

- Suggest a modification to the [BST data structure](#) (week 7 lecture) to implement deleted values.
- Modify the [search algorithm](#) (week 7 lecture) for a "conventional" binary search tree to take into account deleted nodes.
- One significant advantage of deletion-by-marking is that it makes the deletion operation simpler. All that deletion needs to do is search for a node containing the value to be deleted. If it finds such a node, it simply "marks" it as deleted. If it does not find such a node, the tree is unchanged.

Write a deletion algorithm that takes a BST  $t$  and a value  $v$  and returns a new tree which does not contain an undeleted node with value  $v$ .

- The most problematic aspect of deletion-by-marking is insertion. If handled naively, the tree grows as if it contains  $n+d$  values, where  $n$  is the number of nodes containing undeleted values and  $d$  is the number of nodes containing deleted values. If many values are deleted, then the tree becomes significantly larger than necessary.

A more careful approach to insertion can help to limit the growth of the tree by re-using nodes containing deleted values. Modify the algorithm for [AVL tree insertion](#) from the lecture to re-use deleted nodes where possible without causing an imbalance.

**Answer:**

- The simplest modification is to add another field to the node to indicate that the node has been deleted:

```
typedef struct Node {
 bool deleted;
 int data;
 Tree left, right;
} Node;
```

Note that it is not possible to come up with a "distinguished" value to put in the value field to represent "deleted", since all possible values of value are valid.

- The following algorithm ignores nodes that have been deleted:

```
TreeSearch(tree,item):
| Input tree, item
| Output true if item found in tree, false otherwise
|
| if tree is empty then
| return false
| else if item<data(tree) then
| return TreeSearch(left(tree),item)
| else if item>data(tree) then
| return TreeSearch(right(tree),item)
| else // found => check if node has been deleted
| if tree.deleted return false
| else return true
| end if
```

- The following simple function implements deletion-by-marking:

```
TreeDelete(t,v):
| Input tree t, value v
| Output t with v deleted
|
| if t is not empty then // nothing to do if tree is empty
| if v<data(t) then // delete v in left subtree
| left(t)=TreeDelete(left(t),item)
| else if v>data(t) then // delete v in right subtree
| right(t)=TreeDelete(right(t),item)
| else // mark 't' as deleted
| t.deleted=true
| end if
| end if
| return t
```

d. The algorithm below implements the following strategy:

- if the value is already in the AVL tree, the tree is unchanged
- if there are no deleted nodes on the insertion path, insert as a leaf as normal and rebalance if necessary
- if a deleted node is found containing the value to be inserted, un-mark it
- if a deleted node is on the insertion path and if the value to be inserted is between the inorder predecessor and the inorder successor of the deleted node, then replace the value in that node by the new value, and un-mark it

```

min(t):
| Input tree t
| Output minimum value in t
|
| while left(t) not empty do
| t=left(t)
| return data(t)

max(t):
| Input tree t
| Output maximum value in t
|
| while right(t) not empty do
| t=right(t)
| return data(t)

insertAVL(t,v):
| Input tree t, value v
| Output t with item AVL-inserted
|
| if t is empty then
| return new node containing v
| else if v=data(t) then
| return t
| else if t.deleted and max(left(t))<v<min(right(t)) then
| data(t)=v
| t.deleted=false
| else
| if v<data(t) then
| left(t)=insertAVL(left(t),v)
| else if v>data(t) then
| right(t)=insertAVL(right(t),v)
| end if
| if height(left(t))-height(right(t)) > 1 then
| if v>data(left(t)) then
| left(t)=rotateLeft(left(t))
| end if
| t=rotateRight(t)
| else if height(right(t))-height(left(t)) > 1 then
| if v<data(right(t)) then
| right(t)=rotateRight(right(t))
| end if
| t=rotateLeft(t)
| end if
| return t
| end if

```

6. (2-3-4 trees)

Show how a 2-3-4 tree would be constructed if the following values were inserted into an initially empty tree in the order given:

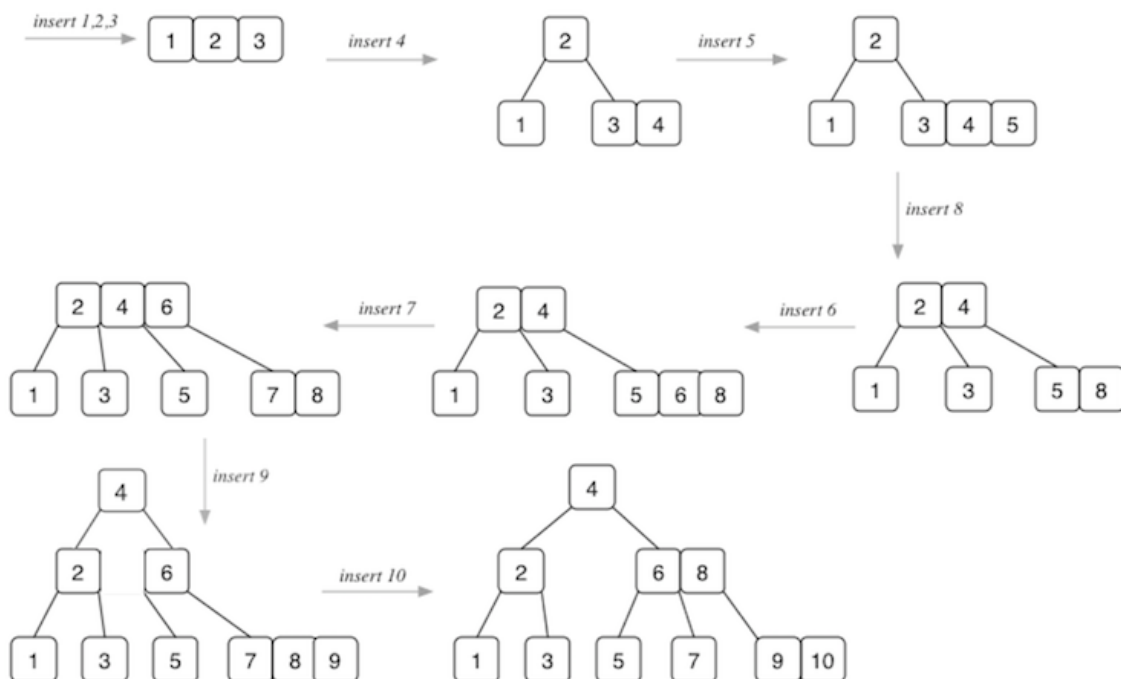
1 2 3 4 5 8 6 7 9 10

Once you have built the tree, count the number of comparisons needed to search for each of the following values in the tree:

1 7 9 13

**Answer:**

The following diagram shows how the tree grows:



Search costs (for tree after insertion of 10):

- search(1): cmp(4),cmp(2),cmp(1)  $\Rightarrow$  cost = 3
- search(7): cmp(4),cmp(6),cmp(8),cmp(7)  $\Rightarrow$  cost = 4
- search(9): cmp(4),cmp(6),cmp(8),cmp(9)  $\Rightarrow$  cost = 4
- search(13): cmp(4),cmp(6),cmp(8),cmp(9),cmp(10)  $\Rightarrow$  cost = 5

#### 7. (Red-black trees)

- a. Show how a red-black tree would be constructed if the following values were inserted into an initially empty tree in the order given:

1 2 3 4 5 8 6 7 9 10

Once you have built the tree, compute the cost (#comparisons) of searching for each of the following values in the tree:

1 7 9 13

- b. Consider the following high-level description from the lecture of an algorithm for inserting items into a red-black tree:

```
insertRB(tree,item,inRight):
 if tree is empty then
 return newNode(item)
 else if item=data(tree) then
 return tree
 end if
 if left(tree) and right(tree) are RED then
 split 4-node
 end if
 recursive insert, re-arrange links/colours after insert
 return modified tree

insertRedBlack(tree,item):
 tree=insertRB(tree,item,false)
 color(tree)=BLACK
 return tree
```

Implement this algorithm in the Red-Black Tree ADT ([RBTree.h](#), [RBTree.c](#)) from the lecture as the function:

```
Tree TreeInsert(Tree t, Item it) { ... }
```

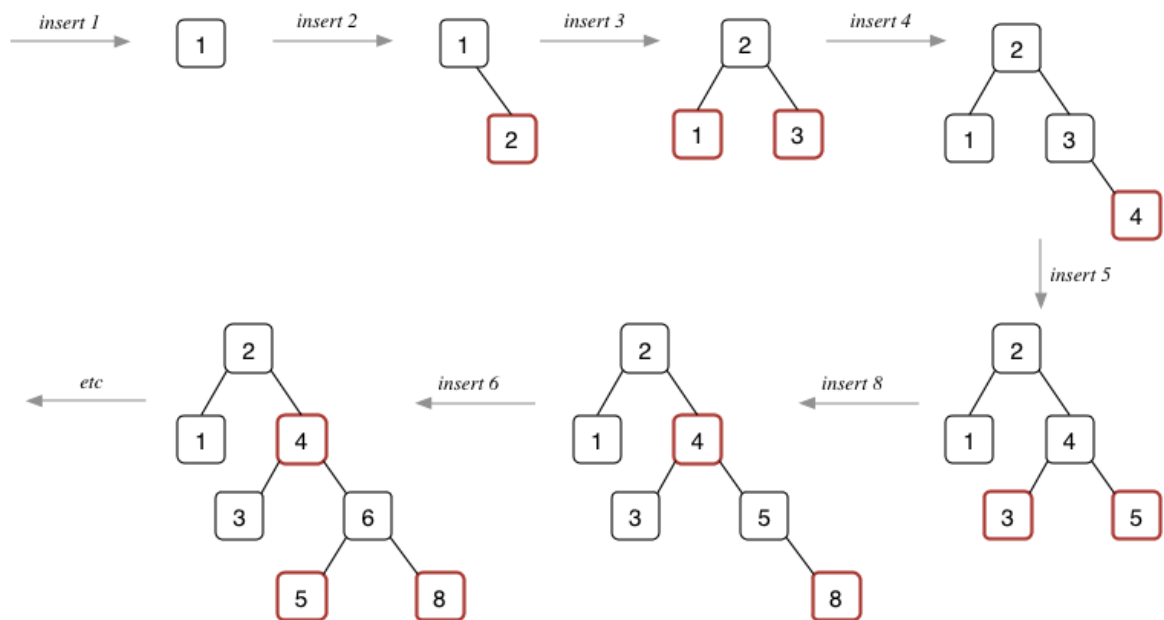
We have created a script that can automatically test your program. To run this test you can execute the dryrun program that corresponds to this exercise. It expects to find the file named `RBTree.c` in the current directory with your implementation of the function `TreeInsert()`.

You can use dryrun as follows:

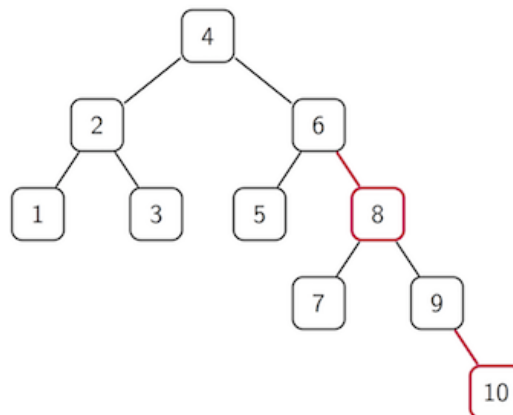
```
prompt$ 9024 dryrun RBTree
```

**Answer:**

a. The following diagrams shows how the tree grows:



Search costs: The number of comparisons needed to search for a node is 1 + the level of the node in the final red-black tree:



- search(1): cost = 3
- search(7): cost = 4
- search(9): cost = 4
- search(13): cost = 5

Note that the resulting red-black tree corresponds to the 2-3-4 tree in Exercise 6, with identical search costs.

b.

```

Tree insertRB(Tree t, Item it, bool inRight) {
 if (t == NULL)
 return newNode(it);
 if (it == data(t))
 return t;
 if (isRed(left(t)) && isRed(right(t))) {
 color(t) = RED;
 color(left(t)) = BLACK;
 color(right(t)) = BLACK;
 }
 if (it < data(t)) {
 left(t) = insertRB(left(t), it, false);
 if (isRed(t) && isRed(left(t)) && inRight) {
 t = rotateRight(t);
 }
 if (isRed(left(t)) && isRed(left(left(t)))) {
 t = rotateRight(t);
 color(t) = BLACK;
 color(right(t)) = RED;
 }
 } else {
 right(t) = insertRB(right(t), it, true);
 if (isRed(t) && isRed(right(t)) && !inRight) {
 t = rotateLeft(t);
 }
 if (isRed(right(t)) && isRed(right(right(t)))) {
 t = rotateLeft(t);
 }
 }
}

```

```

 color(t) = BLACK;
 color(left(t)) = RED;
 }
}
return t;
}

Tree TreeInsert(Tree t, Item it) {
 t = insertRB(t, it, false);
 color(t) = BLACK;
 return t;
}

```

## 8. Challenge Exercise

Extend the BST ADT from the lecture ([BST.h](#), [BST.c](#)) by an implementation of the function

```
deleteAVL(Tree t, Item it)
```

to properly delete an element from an AVL tree (as opposed to lazy deletion, cf. Exercise 5) while maintaining balance.

**Answer:**

The following algorithms implements standard BST deletion and then repairs any imbalances in the same ways as AVL insertion does.

```

Tree AVLrepair(Tree t) {
 int hL = TreeHeight(left(t));
 int hLL = TreeHeight(left(left(t)));
 int hLR = TreeHeight(right(left(t)));
 int hR = TreeHeight(right(t));
 int hRL = TreeHeight(left(right(t)));
 int hRR = TreeHeight(right(right(t)));
 if (hL-hR > 1) {
 if (hLR-hLL > 1)
 left(t)=rotateLeft(left(t)); // left-right case
 t = rotateRight(t);
 } else if (hR-hL > 1) {
 if (hRL-hRR > 1)
 right(t)=rotateRight(right(t)); // right-left case
 t = rotateLeft(t);
 }
 return t;
}

Tree deleteAVL(Tree t, Item it) {
 if (t != NULL) {
 if (it < data(t)) {
 left(t) = TreeDelete(left(t), it);
 t = AVLrepair(t);
 } else if (it > data(t)) {
 right(t) = TreeDelete(right(t), it);
 t = AVLrepair(t);
 } else {
 Tree new;
 if (left(t) == NULL && right(t) == NULL)
 new = NULL;
 else if (left(t) == NULL) // if only right subtree, make it the new root
 new = right(t);
 else if (right(t) == NULL) // if only left subtree, make it the new root
 new = left(t);
 else { // left(t) != NULL and right(t) != NULL
 new = joinTrees(left(t), right(t));
 new = AVLrepair(new);
 }
 free(t);
 t = new;
 }
 }
 return t;
}

```

## Assessment

Group submissions will not be allowed. Your program must be entirely your own work. Plagiarism detection software will be used to compare all submissions pairwise.

Submit your solutions to **Exercise 1 and 7** using the following **give** command:

```
prompt$ give cs9024 week8 BST.c RBTree.c
```

Make sure you spell the filenames correctly. You can run **give** multiple times. Only your last submission will be marked.

The deadline for submission is **Monday, 11 November 11:11:11am**.

Each program is worth 1 mark. Total marks = 2. Auto-marking will be run by the lecturer several days after the submission deadline using different test cases than `dryrun` does.

*Hints:*

- Programs will not be manually marked.
- It is important that the output of your program follows exactly the format shown above, otherwise auto-marking will result in 0 marks.
- Ensure that your program compiles on a CSE machine with the standard options `-Wall -Werror -std=c11`. Programs that do not compile will receive 0 marks.
- `dryrun` and auto-marking also check the correct usage of dynamic memory allocation and pointers as well as the absence of memory leaks.
- Do your own testing in addition to running `dryrun`.